Mid Phase Report — Project Xceed Seat Belt Detection System with Safety Analytics Integration

To: Designated MSIL Project Mentor — Muskan Chojer

From: Dakshayani Sharma — B.Tech IT, 3rd Year, MIT MAHE

Date: May 15, 2026

1. Hardware Architecture

I'm happy to report that all the hardware has been procured and is now in hand. The system is designed around a single-board edge architecture, where the Raspberry Pi 5 handles the entire pipeline on its own — no internet connection is needed at any point. The active cooler has also been physically installed on the Pi.

1.1 Hardware Components

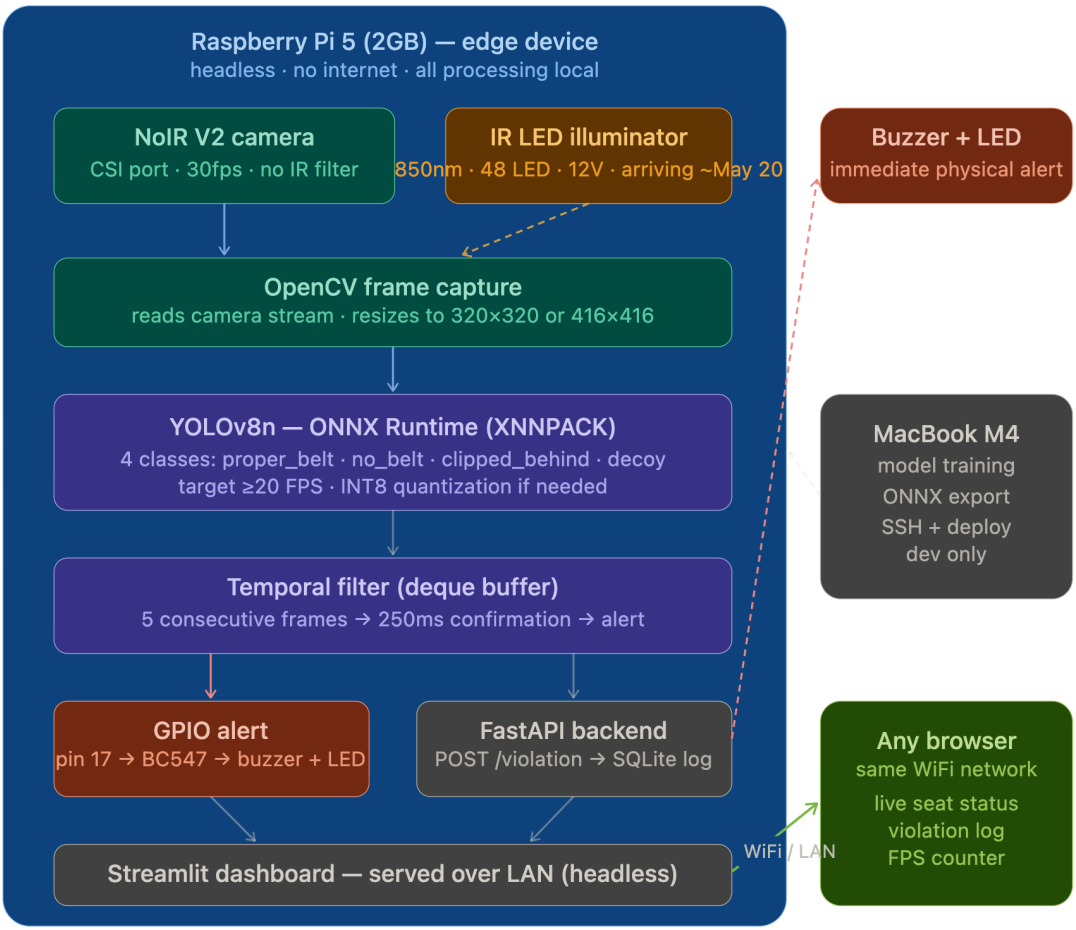
Component	Role in System	Status	Note
Raspberry Pi 5 (2GB RAM)	Primary edge compute unit — runs OS Lite, YOLOv8n ONNX inference, FastAPI backend, Streamlit dashboard	**Received**	Active cooler installed
RPi Camera Module V2 NoIR	Image acquisition — no IR filter enables low-light sensitivity required for <1 lux passive cabin testing	**Received**	FFC cable attached
22-to-15 pin FFC Cable (200mm)	Connects Camera V2 NoIR to Pi 5 CSI port — mandatory adapter as Pi 5 uses 22-pin connector	**Received**	
Official 27W USB-C Power Supply	Delivers 5.1V/5A via USB-PD — prevents CPU throttling during sustained AI inference	**Received**	
Official RPi 5 Active Cooler	PWM fan + heatsink — prevents thermal throttling during continuous inference workloads	**Installed**	Mounted on board
32GB MicroSD Card (A2, 64-bit OS)	System storage — OS, model weights, scripts, SQLite database	**Received**	Pi OS 64-bit
Active Buzzer (5V)	Physical non-compliance alert — triggered via GPIO through BC547 transistor switching circuit	**Received**	

Component	Role in System	Status	Note
BC547 Transistors, Resistors (220Ω, 1kΩ), LEDs, Breadboard, Jumper Wires	GPIO alert circuit — transistor protects Pi GPIO rail; resistors limit current to LED and transistor base	**Received**	
12V 1A DC Adapter	Powers the IR illuminator board independently of the Pi power circuit	**Received**	
IR LED Illuminator (850nm, 48 LED)	Provides near-IR illumination for NoIR camera in Nighttime Passive Cabin (<1 lux) testing	**Ordered**	ETA ~May 20

1.2 System Architecture

The entire pipeline runs on the Raspberry Pi 5 in headless mode. The Streamlit dashboard is served over the local WiFi network, so it can be accessed from any browser on the same network. At no point during inference or logging does the system rely on external services, cloud APIs, or an internet connection.

Layer	Function	Technology
Capture	NoIR V2 camera streams frames at up to 30fps via CSI port	libcamera + OpenCV VideoCapture
Inference	YOLOv8n ONNX model classifies each frame into one of four semantic classes	ONNX Runtime (CPU, XNNPACK backend)
Temporal Filter	Alert triggered only after N consecutive non-compliant frames — suppresses transient false positives	Python deque-based frame buffer (N=5, ~250ms at 20 FPS)
Alert	GPIO pin goes HIGH → BC547 switches → 5V buzzer and warning LED fire immediately	RPi.GPIO library
Logging	Every violation logged: timestamp, seat position, detected class, confidence score	FastAPI + SQLite (aiosqlite)
Dashboard	Live seat compliance status, violation history log, real-time FPS counter — headless, browser accessible	Streamlit (served over LAN, no monitor required)



Seat Belt Detection System — Full Pipeline

2. Proposed Software Algorithms

2.1 Detection Model — YOLOv8n

I chose YOLOv8n (Nano) as the detection backbone after thinking through the three main constraints of this project: inference speed (at least 20 FPS on edge hardware), model size (it needs to fit within 2GB RAM alongside the OS and backend), and detection accuracy at close range inside a vehicle cabin.

- **Speed:** With ONNX Runtime and XNNPACK optimization, YOLOv8n should be able to hit 20+ FPS on the Raspberry Pi 5 at 320×320 or 416×416 input resolution. I'll confirm and document the actual numbers in the Final Report once benchmarking is done.
- **Size:** The model weights come in at roughly 6MB. When you factor in OpenCV, ONNX Runtime, and the OS, total runtime memory stays comfortably under 700MB on the 2GB Pi.
- **Accuracy:** Even though it's the smallest YOLO variant, YOLOv8n still has enough precision for close-range in-cabin detection, since the belt and upper body take up a large portion of the frame.

2.2 Four-Class Semantic Detection Strategy

Rather than sticking to the simple binary classification (belt / no belt) that most existing seatbelt detection systems use, this project defines four semantically distinct classes:

Class	Definition	Why it matters
proper_belt	Belt worn correctly across chest and lap, buckled into restraint	Baseline compliant state — system remains silent
no_belt	Passenger present with no visible belt	Primary non-compliance scenario — triggers immediate alert
clipped_behind	Belt buckled but routed behind the passenger — appears compliant visually	Most dangerous edge case — passenger appears safe but has no protection
decoy	Visual objects resembling a belt: bag straps, cables, lanyards, jacket zippers, scarves	Prevents false negatives — system must not mistake bag strap for a real belt

The **clipped_behind** and **decoy** classes are what make this project stand out technically. Most published systems only use two classes. Building in explicit training for these real-world edge cases is, in my view, both the main engineering challenge and the core contribution of this work.

2.3 Temporal False-Positive Suppression

A deque-based frame buffer keeps track of the last N inference results. The GPIO alert only fires once the buffer has at least 5 consecutive non-compliant frames in a row. At 20 FPS, this works out to a 250ms confirmation window before any alert goes off. This prevents single-frame misclassifications from triggering false alerts, which is especially important for the decoy class where brief occlusions or lighting shifts could otherwise cause spurious triggers.

2.4 ONNX Optimization Pipeline

- **Stage 1 — Training (MacBook M4):** YOLOv8n is trained in PyTorch on the Mac, and the M4 Neural Engine takes care of training without needing a GPU.
- **Stage 2 — ONNX Export:** The best.pt file is exported to ONNX format, which strips out the training overhead and optimizes the compute graph for inference-only use.
- **Stage 3 — Quantization (conditional):** If inference FPS drops below 20 at the target resolution, INT8 quantization will be applied via onnxruntime.quantization, which cuts the model size down by roughly 4× with very little accuracy loss.

2.5 Dataset Strategy

Training data is a mix of three public Roboflow datasets and a custom-captured dataset that I built specifically to cover the edge cases in this project:

- **Public datasets:** I downloaded, extracted, and organized three Roboflow YOLOv8 datasets covering the standard proper_belt and no_belt classes across a range of conditions. Combined, these give around 5,000+ source images.
- **Custom dataset (completed):** I personally captured 133 images using an OpenCV-based webcam capture utility, spread across all four classes. The images cover different shirt colors, lighting conditions, seating angles, and head poses. I can always add more custom images later during model testing if needed.
- **Augmentation strategy:** Augmentation includes horizontal flip, brightness/contrast variation of ±30%, Gaussian noise, and mosaic, all applied through the Roboflow augmentation pipeline to simulate the kind of variability you'd see in a real car cabin.

3. Current Project Progress

3.1 Completed

- **Hardware procurement:** All components received. Active cooler installed on Raspberry Pi 5. Hardware fully assembled and ready for OS setup.
- **Software environment:** Ultralytics YOLOv8, OpenCV, ONNX Runtime, FastAPI, Streamlit, and all dependencies installed and verified on MacBook M4 development machine.
- **YOLO environment verification:** yolo checks command confirmed setup complete — Python 3.9.6, torch 2.8.0, all required packages passing. YOLOv8n weights downloaded. Training pipeline verified with initial test run generating runs/ directory.
- **Project repository:** GitHub repository initialized (dakshayani-codes/project-xceed-msil), modular folder structure committed, .gitignore configured for large files.
- **Public dataset collection:** Three Roboflow YOLOv8 seatbelt datasets downloaded, extracted, and organized into datasets/public/ with train/valid/test splits.
- **Custom dataset collection:** 133 images captured across all four classes using OpenCV webcam capture utility (capture_custom.py). Covers proper_belt (~52), no_belt (~34), decoy (~20), clipped_behind (~27) with lighting and pose variations.
- **Dataset configuration:** ai/dataset.yaml created defining all four classes and dataset paths for YOLOv8 training pipeline.
- **Capture pipeline:** ai/capture_custom.py implemented — OpenCV-based webcam capture script with SPACE to capture, auto class-wise folder saving.
- **Project structure:** Modular directory structure established: ai/, backend/, frontend/, hardware/, datasets/, reports/, demo/.
- **README documentation:** README.md written covering hardware stack, software stack, project structure, detection classes, and planned workflow.

3.2 In Progress

- **Dataset annotation:** Bounding box annotation of custom images in YOLO format using LabelImg. Partially completed — continuing in parallel with training preparation.
- **Model training:** YOLOv8n training on merged public + custom dataset — starting immediately on MacBook M4.
- **Raspberry Pi OS setup:** Pending SD card reader procurement (required to flash/configure OS). All WiFi configuration files prepared. Pi physically ready.

3.3 Pending

- ONNX model export and INT8 quantization
- Raspberry Pi deployment and FPS benchmarking
- GPIO circuit assembly and buzzer/LED integration test
- FastAPI + Streamlit deployment on Pi headless
- Multi-lighting scenario testing (IR board arriving ~May 20)
- Temporal filter integration and tuning
- Final demo video recording and Technical Final Report

4. Challenges Encountered

4.1 Hardware Procurement — Market Constraints

Finding a Raspberry Pi 5 turned out to be harder than expected. Stock was quite limited across Indian vendors throughout April 2026, with prices ranging from ₹12,149 to ₹15,108 — well over the standard MRP. I eventually managed to source the 2GB variant from Thingbits at ₹7,287 incl. GST, which was the most affordable option I could find. The Camera Module 3 NoIR was also out of stock at multiple vendors, so I went with the NoIR V2 as an alternative — it's technically equivalent and meets the <1 lux requirement at a lower price.

4.2 Academic Commitments

Most of the hardware procurement, architecture design, and dataset planning had to be done alongside end-semester exams at MIT MAHE, which slowed things down a bit. Once the exams were over, I was able to shift fully to software implementation and integration. The revised timeline takes this into account, and I'm still confident the June 2nd final review deadline is achievable.

4.3 Technical Challenges — Identified and Planned For

- **Low-contrast detection:** A black belt on dark clothing gives the model very little contrast to work with. To handle this, I'm using targeted augmentation with brightness/contrast variation, and I'll also look into whether a two-stage pipeline (upper body detection first, then belt state classification) does better on low-contrast cases.
- **Decoy discrimination:** Bag straps and lanyards look surprisingly similar to a seatbelt, especially when worn diagonally. The custom decoy dataset is my main line of defense

here. I plan to test the model against unseen decoy objects and will document any failure cases honestly in the Final Report.

- **Clipped-behind detection:** Depending on camera angle, a clipped-behind belt can look almost identical to a properly worn one — which makes this case particularly tricky. I've captured this scenario from multiple angles in the custom dataset, and camera placement in the final prototype will be chosen to give the best view of this state.
- **FPS target on 2GB Pi:** Running inference, FastAPI, and Streamlit all at the same time on a 2GB device means memory management has to be tight. To deal with this, Streamlit is configured with a 1-second polling interval, inference and the backend run as separate processes, and I'll do memory profiling before adding each new component.
- **IR illuminator delay:** The IR board is arriving around May 20, which leaves about 12 days for passive cabin (<1 lux) testing. To make the most of that window, I'm aiming to finish all other testing before it arrives so the <1 lux tests can be prioritized once it's in hand.
- **Single camera FOV limitation:** A single camera can't cover the front, rear, and third-row passengers at the same time without repositioning. That said, the architecture is designed to be multi-camera extensible, and the current prototype shows the single-camera pipeline working end-to-end — the same approach can be replicated per seat zone if needed.

5. Dataset Architecture

5.1 Public Datasets

Dataset	Source	Images (approx.)	Classes
seatbelt_v1	Roboflow Universe — seatbelttraining	~3,489	person-seatbelt, person-noseatbelt
seatbelt_v2	Roboflow Universe — fyp-atxxb	~900	seatbelt, non-seatbelt
seatbelt_v3	Roboflow Universe — karan-panja	~660	seatbelt, no-seatbelt

5.2 Custom Dataset — Captured in This Project

Class	Images	Capture variations
proper_belt	~52	Different shirt colors (white, black, grey), multiple seating angles, frontal and side view, bright and low light, different head poses. Bag strap used diagonally to simulate realistic belt orientation.
no_belt	~34	Frontal view, tilted head, varied postures, different lighting conditions, different background contexts.

Class	Images	Capture variations
decoy	~20	Bag straps, charger cables, diagonal accessories worn across chest — simulating common false-positive objects.
clipped_behind	~27	Belt buckled but routed behind the seated person — captured from standard camera angle to evaluate visibility of this deceptive state.
TOTAL	~133	Across all four project-specific classes. Annotation in YOLO format in progress.

6. Revised Implementation Timeline

Period	Status	Key Activities
Apr 6 – Apr 12	**Done**	Initial Plan Report submitted. Hardware list approved. Procurement initiated.
Apr 13 – Apr 30	**Done**	All hardware procured and received. System architecture finalized. Software environment configured on Mac. GitHub repository initialized.
May 1 – May 14	**Done**	Public datasets downloaded and organized. Custom dataset of 133 images captured across 4 classes. YOLOv8 environment verified. dataset.yaml created. Project structure established.
May 15 (Today)	**Today**	Mid Progress Report submission. Pi OS setup initiated. Model training begins on Mac.
May 16 – May 22	**Upcoming**	Complete Pi OS setup and SSH configuration. Camera + GPIO testing on Pi. YOLOv8n training on Mac — first model checkpoint. ONNX export and initial Pi deployment.
May 23 – May 31	**Upcoming**	FPS benchmarking on Pi. IR illuminator integration (~May 20). Multi-lighting scenario testing. FastAPI + Streamlit on Pi. Temporal filter integration and tuning.
Jun 1 – Jun 2	**Upcoming**	Final end-to-end demo. Video recording. Technical Final Report submission. Final Review.

7. Key Design Decisions and Justifications

Decision	Rationale	Alternative considered
YOLOv8n over MobileNetV3	End-to-end detection + classification in single pass; strong community support; Ultralytics ONNX export directly supported	MobileNetV3 + custom classifier (two-stage) — discarded due to added pipeline complexity
2GB Pi + Mac for dashboard build	Keeps Pi RAM usage under 700MB; React build is served as static files requiring only ~20MB on Pi;	4GB Pi with React running natively — rejected due to cost (₹12,000+) and stock unavailability

Decision	Rationale	Alternative considered
	MacBook used only for build step not during demo	
Streamlit over React for dashboard	Pure Python, runs headlessly on Pi, no Node.js runtime overhead; Muskan explicitly suggested native UI over web framework	React + Vite (evaluated but deprioritized due to JS dependency and higher RAM footprint)
NoIR V2 over Camera Module 3 NoIR	Module 3 NoIR was out of stock across all Indian vendors during procurement window and overly expensive; V2 NoIR meets the <1 lux requirement at lower cost	Camera Module 3 NoIR (preferred technically but unavailable)
Pi GPIO directly for alert (no Arduino)	Eliminates second controller and serial communication overhead; Pi 5 GPIO + BC547 transistor circuit sufficient for buzzer switching	Arduino/LPC1768 as dedicated controller — removed after MSIL mentor feedback
Temporal filter (5 frame buffer)	Prevents single-frame misclassifications from triggering false alerts; 250ms delay at 20 FPS is imperceptible to passengers but eliminates noise	Immediate single-frame triggering — rejected due to high false positive rate